

PROJEKT ZTP - APLIKACJA DO NAUKI JĘZYKÓW

- **Aplikacja desktopowa** napisana w języku JAVA z interfejsem graficznym zrealizowanym przy pomocy biblioteki Swing.

- **Krótki opis aplikacji:** Jest to aplikacja służąca do efektywnej nauki języków obcych i nie tylko, gdyż zestawy słówek mogą zawierać dosłownie wszystko. Program oferuje spersonalizowany system powtórek oraz różnorodne tryby nauki (grywalizacja), dostosowane do poziomu zaawansowania użytkownika (CEFR). Użytkownik może śledzić swoje postępy w nauce dzięki statystykom wyświetlanym po każdej lekcji oraz podglądowi statystyk ogólnych. Program zawiera dodatkowo takie funkcjonalności jak rejestracja, logowanie, zapisywanie postępów dla każdego użytkownika osobno oraz satysfakcjonujący system progresowania oparty na poziomie użytkownika.

- **Użyte technologie:**

Java 24,

Swing,

Wzorce projektowe (GoF).

-**Użyte wzorce projektowe:**

Singleton,

Proxy,

Observer,

Memento,

MVC

- **Skład zespołu:**

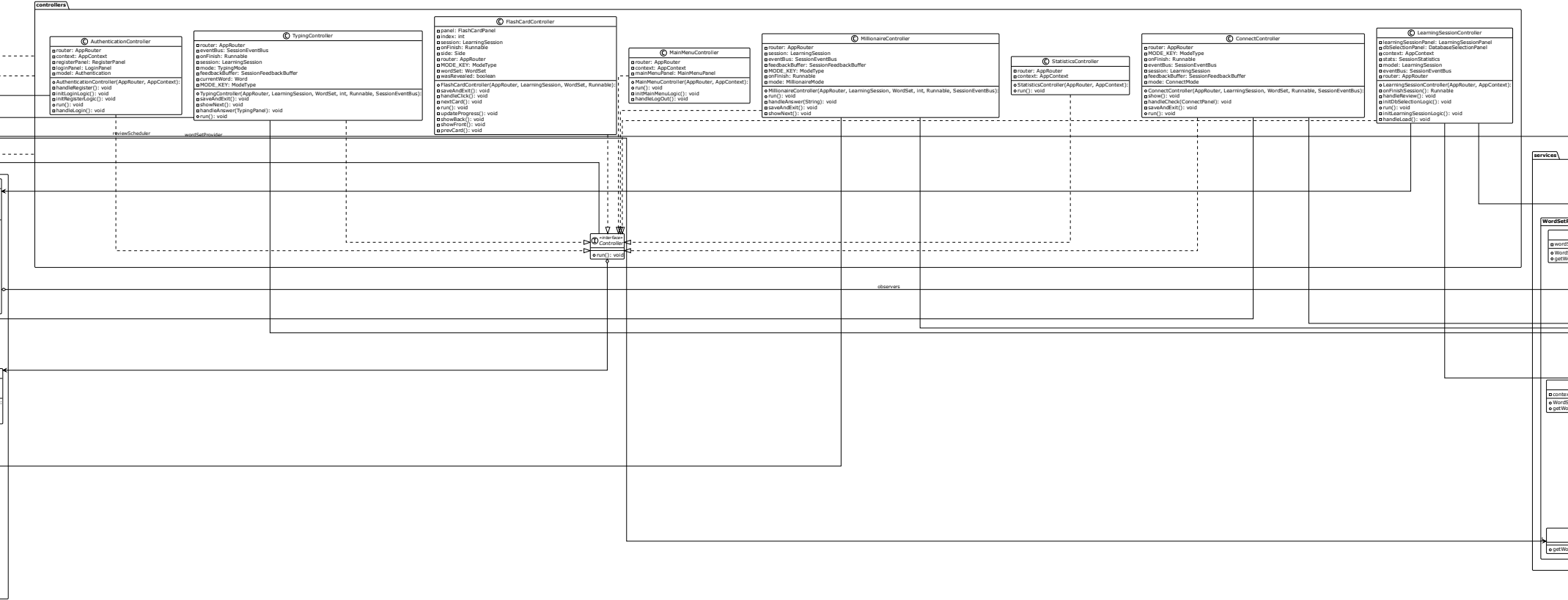
Maciej Balcerzak,

Piotr Borys,

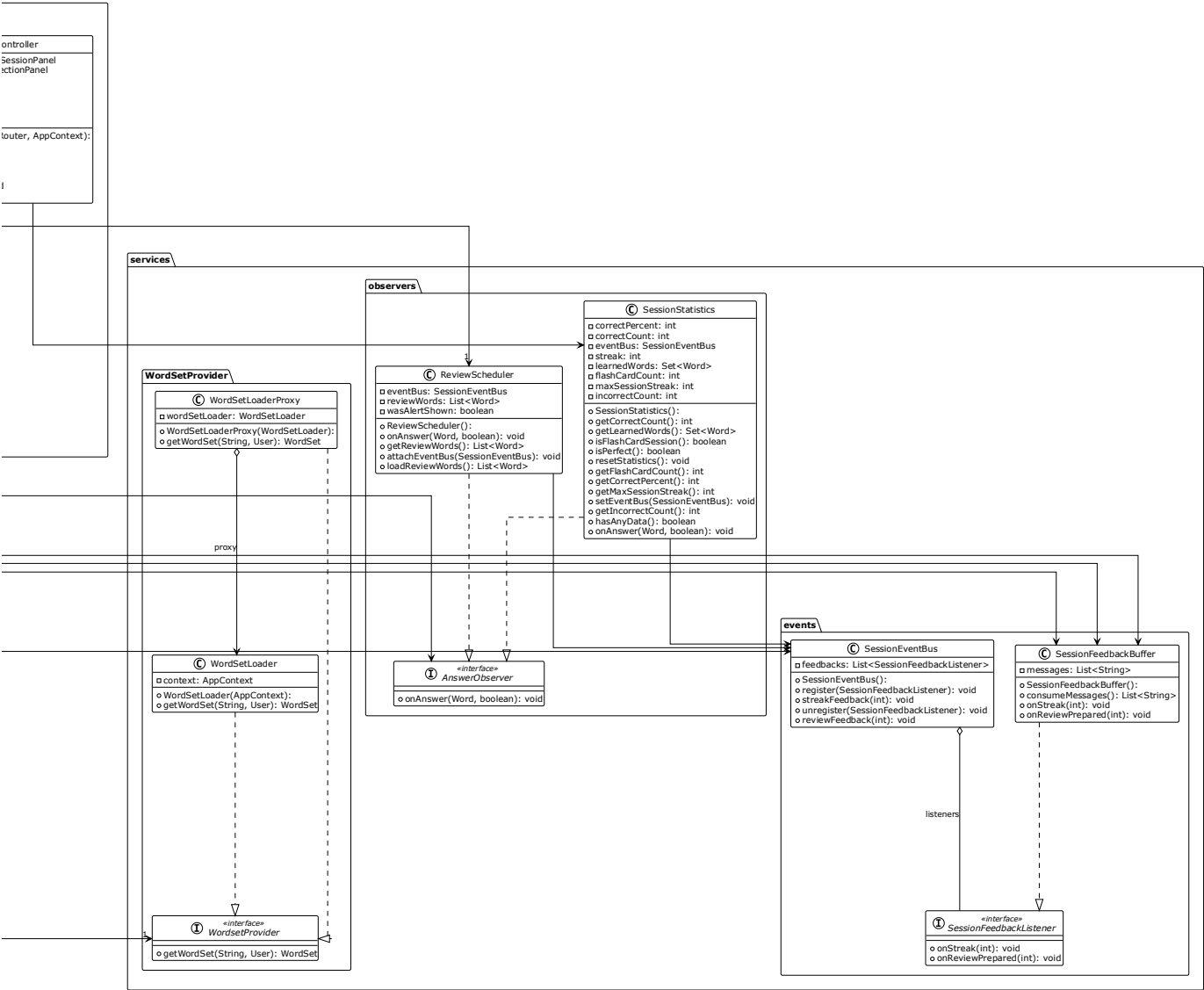
Jan Sztukowski

[illegible]

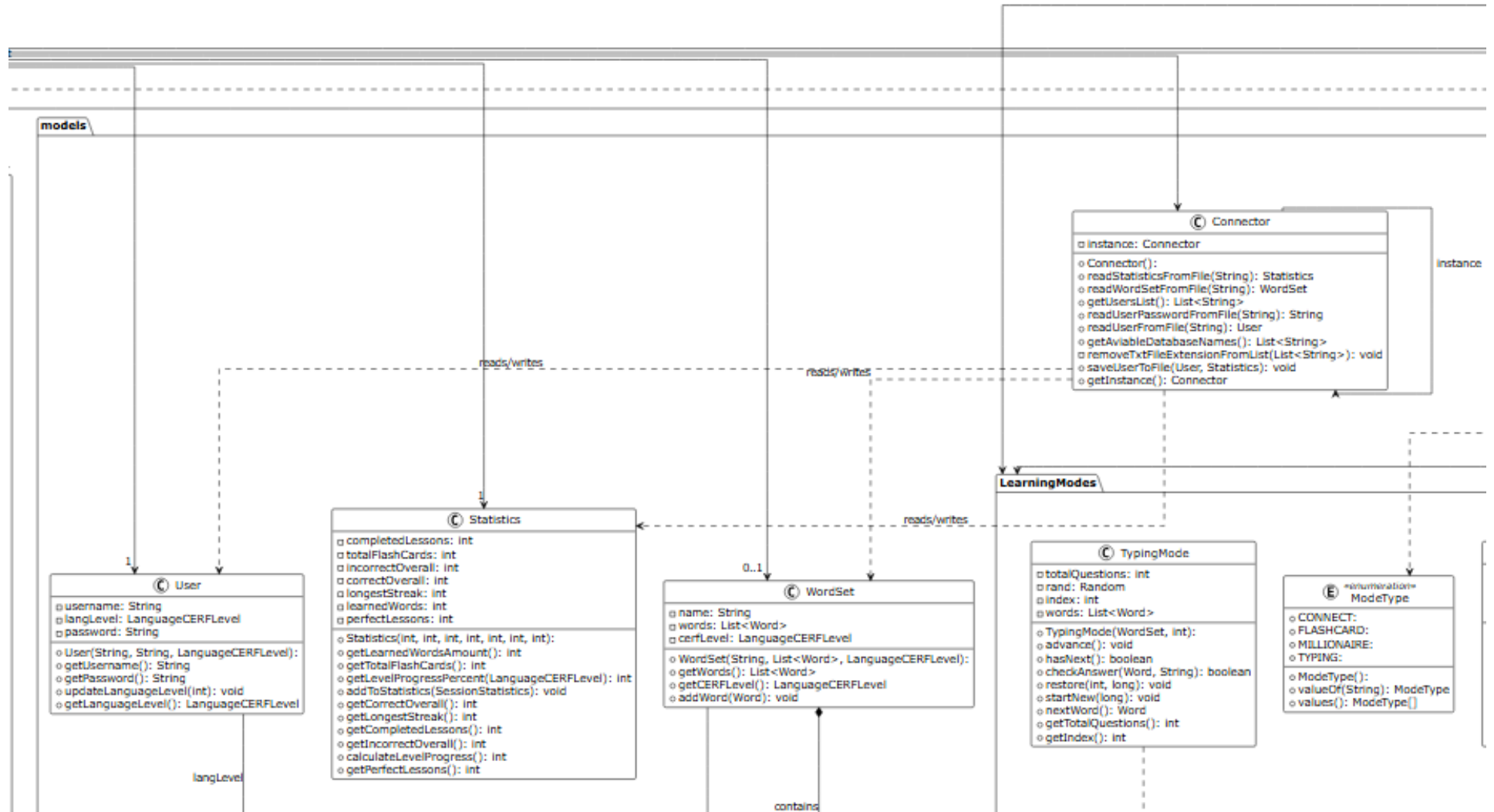
PEŁNY DIAGRAM KLAS BUDUJĄCYCH PROGRAM cz. 2



PEŁNY DIAGRAM KLAS BUDUJĄCYCH PROGRAM cz. 3



WZORCE PROJEKTOWE (GoF): WZORZEC KREACYJNY – SINGLETON



WZORCE PROJEKTOWE (GoF): WZORZEC KREACYJNY – SINGLETON

- CEL UŻYCIA:

Zapewnienie istnienia wyłącznie jednej instancji klasy odpowiedzialnej za operacje na plikach (odczyt/zapis danych użytkownika i zestawów słówek).

Gwarantuje to spójny dostęp do zasobów dyskowych i zapobiega konfliktom, które mogłyby powstać przy próbie jednoczesnego zapisu przez wiele obiektów.

- PRZYPORZĄDKOWANIE KLAS DO WSZYSTKICH RÓL WZORCA:

Singleton - Klasa Connector. Posiada prywatny konstruktor, statyczne pole przechowujące jedyną instancję oraz statyczną metodę fabrykującą getInstance().

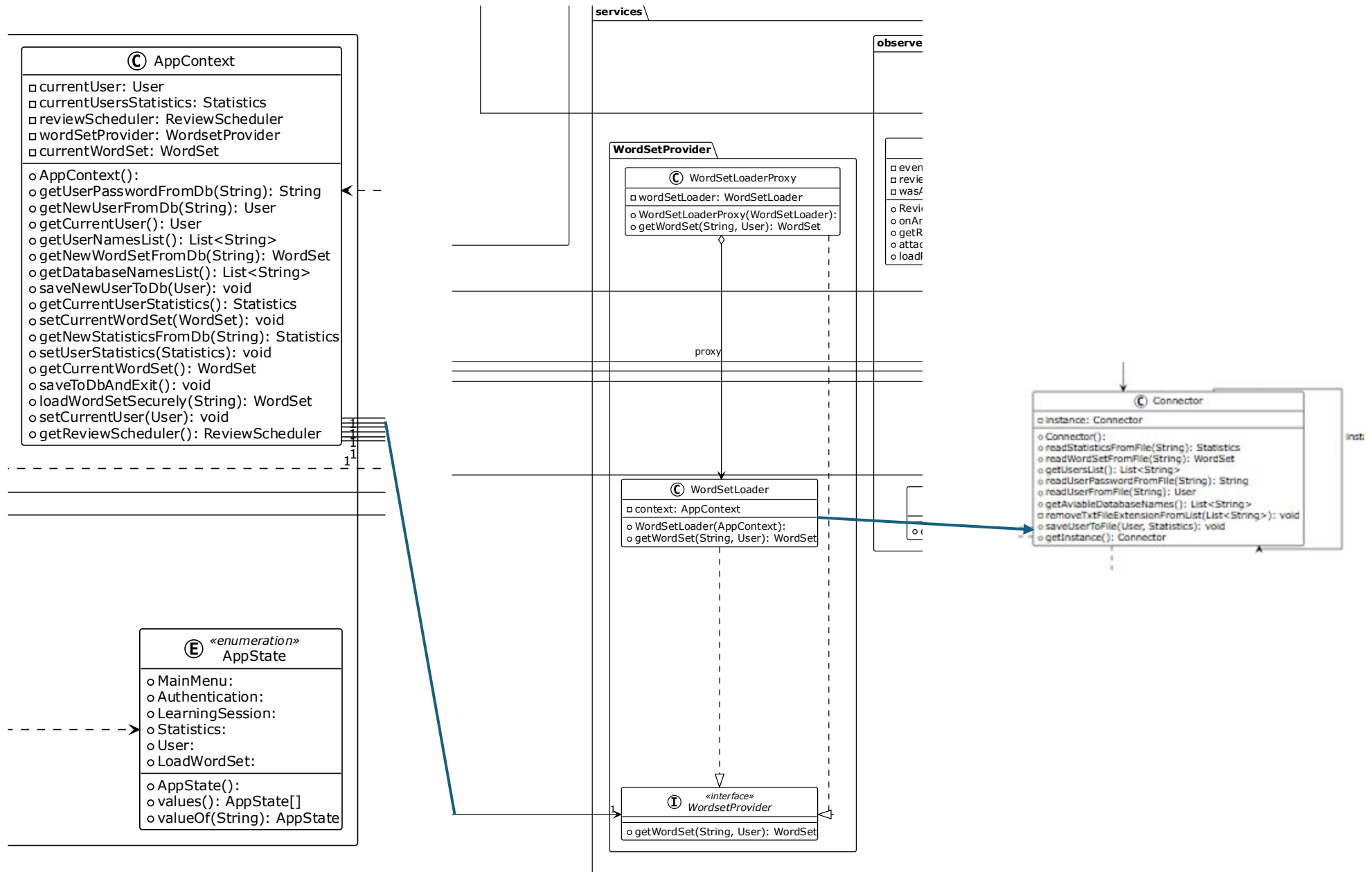
- LOKALIZACJA WZORCA W KODZIE:

Wzorzec znajduje się w pakiecie models w pliku Connector.java. W tym pliku zawierają się wszystkie niezbędne pola i metody definiujące bycie singletonem. Następnie Connector jest używany w klasie AppContext która stanowi główny dostęp programu do danych, a więc i do plików bazy danych.

- WEKTOR ZMIAN:

Centralizacja mechanizmu zapisu danych. Jeśli w przyszłości będzie potrzeba zmiany sposobu przechowywania danych z plików tekstowych (.txt) na np. bazę danych SQL lub format JSON/XML, modyfikacja będzie wymagana wyłącznie wewnątrz klasy Connector. Reszta aplikacji (w tym również AppContext) pozostanie nienaruszona, ponieważ korzysta z tych samych metod publicznych Singletona.

WZORCE PROJEKTOWE (GoF): WZORZEC STRUKTURALNY – PROXY



WZORCE PROJEKTOWE (GoF): WZORZEC STRUKTURALNY – PROXY

- CEL UŻYCIA:

Kontrola dostępu do zestawów słówek przez użytkowników na podstawie poziomu użytkownika. Poszczególne zestawy słówek są odblokowywane dopiero po osiągnięciu odpowiedniego poziomu.

- PRZYPORZĄDKOWANIE KLAS DO WSZYSTKICH RÓL WZORCA:

ServiceInterface: WordsetProvider -interfejs usługi.

Service: WordSetLoader – faktycznie czyta pliki z dysku.

Proxy: WordSetLoaderProxy – pośredniczy w wywołaniu, może sprawdzać uprawnienia.

Client: AppContext – jako główny dostęp do danych całego programu

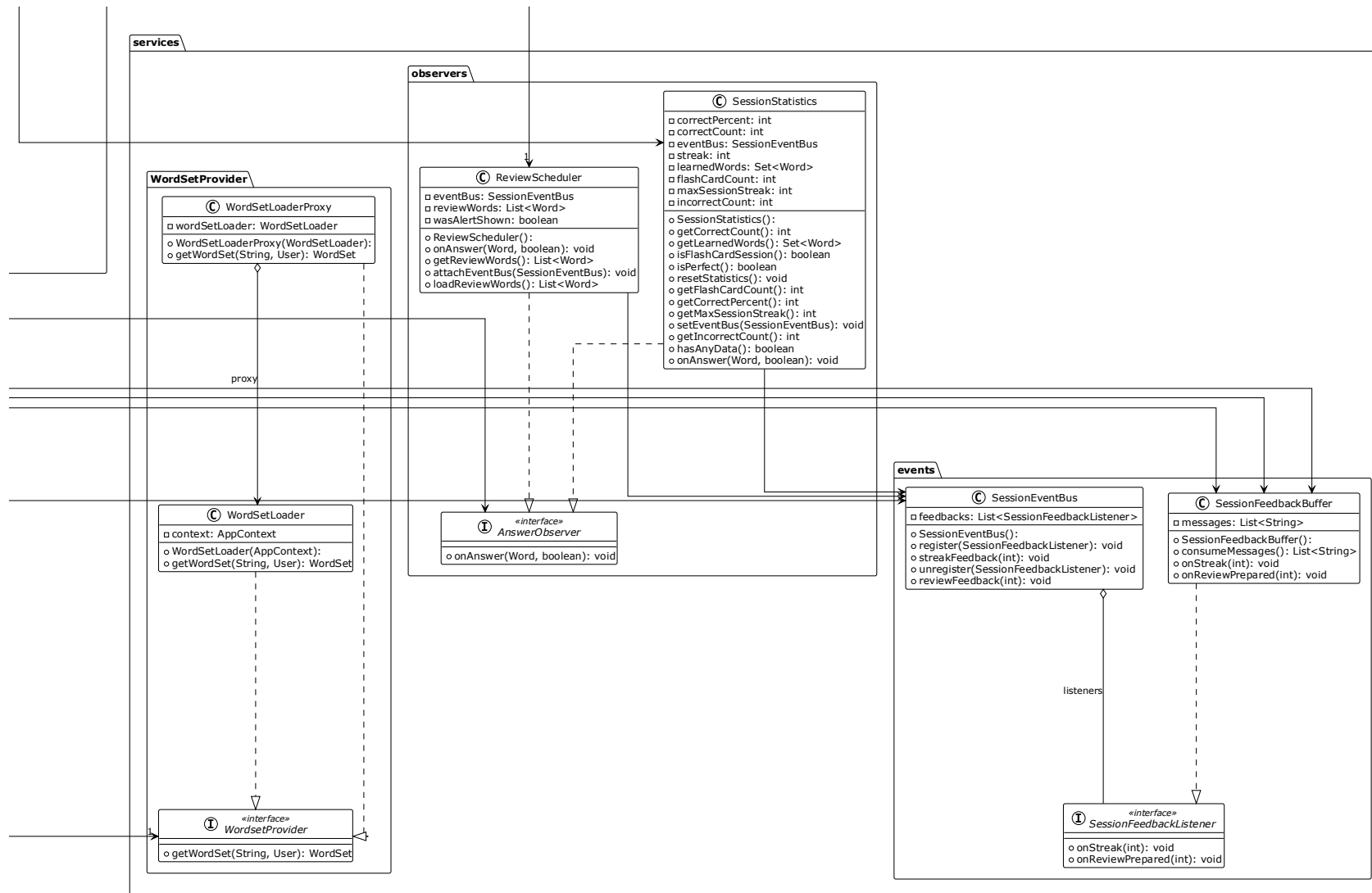
- LOKALIZACJA WZORCA W KODZIE:

Wzorzec znajduje się w pakiecie service, w podpakiecie WordSetProvider w trzech plikach: WordsetProvider.java, WordSetLoader.java oraz WordSetLoaderProxy.java. W tych plikach zawierają się wszystkie niezbędne pola i metody definiujące wzorzec proxy jak wspólna metoda getWordSet(String, User). Następnie WordSetLoaderProxy jest używany poprzez interfejs WordSetProvider w klasie AppContext która stanowi główny dostęp programu do danych, a do jej zadań należy używanie tego pośrednika.

- WEKTOR ZMIAN:

Wprowadzenie szyfrowania plików z bazą słówek lub leniwe ładowanie (lazy loading) dużych zestawów bez zmian w reszcie aplikacji.

WZORCE PROJEKTOWE (GoF): WZORZEC BEHAWIORALNY – OBSERVER



WZORCE PROJEKTOWE (GoF): WZORZEC BEHAWIORALNY – OBSERVER

- CEL UŻYCIA:

Reagowanie na odpowiedzi udzielane przez użytkownika w trakcie sesji nauki.

Oddzielenie logiki trybów nauki od reakcji na odpowiedzi użytkownika pozwala obsługiwać je przez niezależne komponenty aplikacji (np. statystyki sesji) bez bezpośredniego powiązania z mechanizmem rozwiązywania zadań. Umożliwia to dynamiczne dodawanie i usuwanie reakcji na zdarzenia oraz łatwą rozbudowę aplikacji bez modyfikowania istniejącego kodu trybów nauki.

- PRZYPORZĄDKOWANIE KLAS DO WSZYSTKICH RÓL WZORCA:

Publisher – LearningSession - Przechowuje listę obserwatorów oraz powiadamia ich o odpowiedziach użytkownika za pomocą metody `notifyObservers()`.

Subscriber (interfejs) – AnswerObserver - Definiuje metodę `onAnswer(Word w, boolean correct)`.

Subscriber (implementacje) – SessionStatistics, ReviewScheduler - `SessionStatistics` zbiera statystyki bieżącej sesji, natomiast `ReviewScheduler` analizuje błędne odpowiedzi i przygotowuje dane do sesji powtórkowej.

- LOKALIZACJA WZORCA W KODZIE:

Wzorzec znajduje się w pakiecie `services.observers` w postaci:

- interfejsu `AnswerObserver`,
- klas `SessionStatistics` oraz `ReviewScheduler` implementujących interfejs `AnswerObserver`.

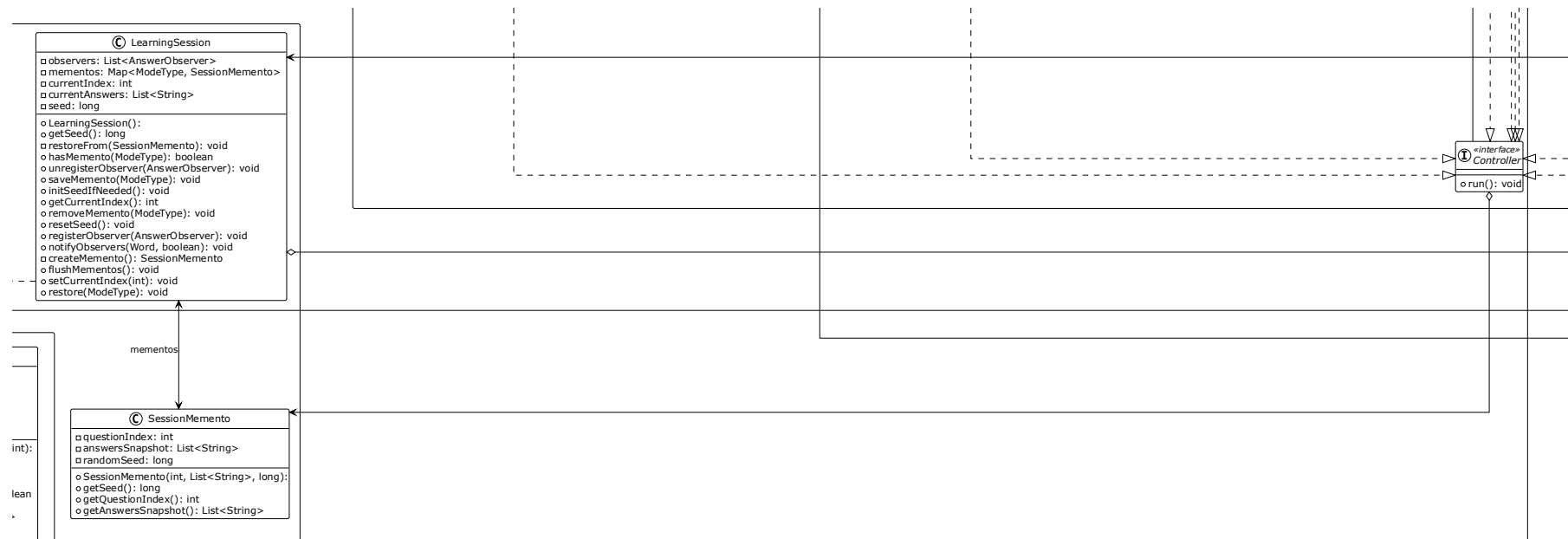
Powiadamanie obserwatorów odbywa się w kontrolerach poszczególnych trybów nauki (np. `MillionaireController`), które po obsłużeniu odpowiedzi użytkownika wywołują w `LearningSession` metodę `notifyObservers()`. Metoda ta umożliwia przekazanie zdarzenia wszystkim zarejestrowanym obserwatorom.

- WEKTOR ZMIAN:

Zastosowanie wzorca `Observer` umożliwia łatwą rozbudowę aplikacji o nowe reakcje na odpowiedzi użytkownika bez modyfikowania istniejącego kodu trybów nauki.

W przyszłości możliwe będzie rozszerzenie aplikacji o nowe obserwatory, np. system osiągnięć, codziennych zadań (questów), adaptacyjny dobór trudności, zapis historii odpowiedzi lub analizę czasu reakcji, poprzez dodanie nowych klas implementujących interfejs `AnswerObserver` i ich rejestrację w sesji nauki.

WZORCE PROJEKTOWE (GoF): WZORZEC BEHAWIORALNY - MEMENTO



- CEL UŻYCIA:

Umożliwienie **zachowania i przywracania stanu sesji nauki** (np. aktualnego indeksu pytania, odpowiedzi użytkownika oraz seedu losowości) **bez naruszania enkapsulacji obiektów trybów nauki**.

Wzorec Memento pozwala użytkownikowi: przerwać tryb nauki, wrócić do menu, a następnie **kontynuować naukę od miejsca, w którym została przerwana**.

Rozwiązanie to jest kluczowe w aplikacji edukacyjnej, gdzie przerwanie sesji nie powinno skutkować utratą postępu.

- PRZYPORZĄDKOWANIE KLAS DO WSZYSTKICH RÓL WZORCA:

• Memento – SessionMemento

Przechowuje migawkę stanu sesji nauki: indeks aktualnego pytania, listę odpowiedzi, seed generatora liczb losowych.

- **Originator** – LearningSession

Odpowiada za: tworzenie obiektów SessionMemento, odtwarzanie własnego stanu na podstawie przekazanego mementa, hermetyzację szczegółów przechowywanego stanu przed innymi komponentami systemu. Dodatkowo LearningSession przechowuje mementa w wewnętrznej strukturze danych EnumMap<ModeType, SessionMemento>, przypisując je do konkretnych trybów nauki.

- **Caretaker** – kontrolery trybów nauki (FlashCardController, MillionaireController, TypingController, ConnectController)
Kontrolery decydują **kiedy zapisać i kiedy przywrócić** stan sesji, nie mając bezpośredniego dostępu do wewnętrznych pól mementa.

W implementacji zastosowano wariant wzorca Memento dostosowany do architektury MVC, w którym kontrolery pełnią rolę Caretakera, natomiast obiekt LearningSession agreguje i odtwarza swój stan jako Originator.

- **LOKALIZACJA WZORCA W KODZIE:**

- **Definicja wzorca:**
 - models/SessionMemento.java – definicja mementa
 - models/LearningSession.java – tworzenie, zapisywanie i przywracanie mementów
- **Użycie wzorca:**
 - controllers/FlashCardController, controllers/MillionaireController, controllers/TypingController, controllers/ConnectController
 - Wywołanie zapisu stanu: `session.saveMemento(MODE_KEY)`
 - Wywołanie przywracania stanu: `session.restore(MODE_KEY)`

Każdy tryb posiada własny klucz (ModeType), co pozwala na **niezależne przechowywanie stanu dla wielu trybów jednocześnie**.

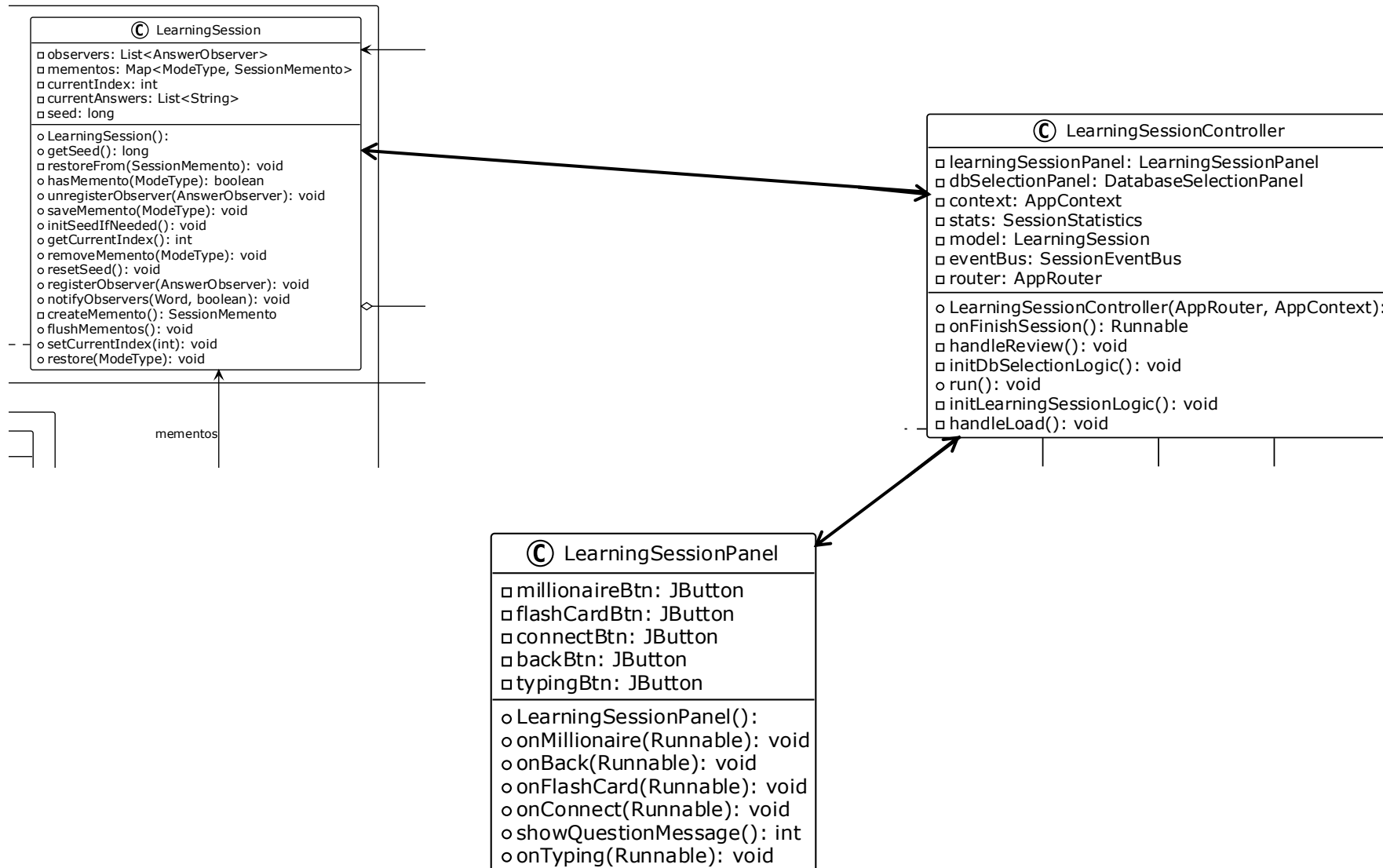
- **WEKTOR ZMIAN:**

Zastosowanie wzorca Memento umożliwia dalszą rozbudowę aplikacji bez ingerencji w istniejącą logikę trybów nauki, np.:

- dodanie nowych trybów nauki bez modyfikowania mechanizmu zapisu stanu,
- rozszerzenie mementa o dodatkowe informacje (np. czas sesji),
- zapis mementów na dysku lub w bazie danych w celu pełnego przywracania sesji po ponownym uruchomieniu aplikacji.

Dzięki wzorcowi Memento zmiany te mogą być realizowane lokalnie, bez naruszania kontraktów klas kontrolerów.

WZORCE PROJEKTOWE (GoF): WZORZEC ARCHITEKTONICZNY - MVC



WZORCE PROJEKTOWE (GoF): WZORZEC ARCHITEKTONICZNY - MVC

- CEL UŻYCIA:

Wydzielenie logiki biznesowej aplikacji, warstwy prezentacji oraz logiki sterującej interakcją użytkownika w celu umożliwienia stworzenia czytelnego, rozszerzalnego i łatwego w utrzymaniu interfejsu graficznego.

Zastosowanie MVC pozwala na rozwój aplikacji edukacyjnej z graficznym interfejsem użytkownika w taki sposób, aby zmiany w wyglądzie interfejsu, logice nauki lub obsłudze zdarzeń użytkownika były od siebie niezależne.

- PRZYPORZĄDKOWANIE KLAS DO WSZYSTKICH RÓL WZORCA:

Model: pakiet **models** - zawiera klasy odpowiedzialne za logikę biznesową aplikacji, przetwarzanie danych oraz stan sesji nauki.

Widok (View): pakiet **views** - zawiera komponenty interfejsu graficznego (Swing), których zadaniem jest prezentacja danych oraz przekazywanie akcji użytkownika do kontrolerów.

Kontroler (Controller): pakiet **controllers** - zawiera klasy pośredniczące pomiędzy widokami a modelami, obsługujące logikę sterującą przebiegiem aplikacji.

- LOKALIZACJA WZORCA W KODZIE:

Wzorzec MVC jest zastosowany globalnie w całym projekcie poprzez wyraźny podział pakietów:

- models – implementacja Modelu (logika biznesowa, dane),
- views – implementacja Widoków (komponenty GUI Swing),
- controllers – implementacja Kontrolerów (obsługa zdarzeń i przepływu aplikacji).

Definicja wzorca wynika bezpośrednio ze struktury projektu. Użycie wzorca widoczne jest m.in. w klasie TypingController, która:

- tworzy widok TypingPanel,
- pobiera dane z modelu TypingModel,
- reaguje na akcje użytkownika i steruje przebiegiem sesji nauki.

- WEKTOR ZMIAN:

Zastosowanie MVC umożliwia łatwą rozbudowę aplikacji bez naruszania istniejącego kodu:

- dodanie nowych trybów nauki wymaga stworzenia nowego kontrolera i widoku bez modyfikacji istniejących modeli,
- zmiana interfejsu graficznego (np. inny układ paneli lub inna biblioteka GUI) nie wpływa na logikę nauki,
- możliwe jest rozszerzenie modelu o historię sesji, statystyki długoterminowe lub synchronizację danych bez ingerencji w warstwę widoku,
- ułatwiona jest praca zespołowa, ponieważ każda warstwa może być rozwijana niezależnie.

ROZWIĄZANIA SPECYFICZNE DLA UŻYTEGO JĘZYKA JAVA, DODATKOWE BIBLIOTEKI

- **Java Swing** (javax.swing): Warstwa Prezentacji (View) w architekturze MVC.

-Wykorzystano menedżera układu CardLayout w klasie MainFrame. Dzięki temu aplikacja posiada tylko jedno główne okno, a poszczególne ekrany są podmieniane dynamicznie jako panele. Eliminuje to chaos związany z otwieraniem wielu okienek pop-up.

- Każdy tryb nauki posiada dedykowany panel rozszerzający JPanel z własnym LayoutManagerem (np. GridBagLayout, BorderLayout).

<https://docs.oracle.com/javase/8/docs/api/javax/swing/package-summary.html>

- **Rekordy:** Klasa Word zaimplementowana jako record (Java 14+ feature).

Cel: Zwięzła definicja niemutowalnego obiektu danych (Data Carrier), redukcja kodu szablonowego (automatyczne getters, equals, hashCode).

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Record.html>

- **Java IO & Streams:** Wczytywanie baz danych i konfiguracji użytkowników oparte na java.io i java.util.stream (przetwarzanie linii tekstu).

Cel: Bardziej zwięzłe funkcje odczytu danych z plików oraz większe możliwości dostosowywania sposobu prezentacji tych danych.

<https://docs.oracle.com/javase/8/docs/api/java/util/stream/Stream.html>

PODZIAŁ PRACY W ZESPOLE

-Maciej Balcerzak

- Implementację wszystkich trybów nauki, w tym:
 - logikę poszczególnych trybów,
 - odpowiadające im kontrolery (tj. FlashCardController, TypingController, MillionaireController, ConnectController),
 - panele interfejsu użytkownika (Swing) dla każdego trybu.
- Zaprojektowanie i implementację mechanizmu sesji nauki, obejmującą:
 - klasę LearningSession,
 - kontroler sesji nauki,
- Implementację wzorca projektowego Memento, w tym:
 - Klasę SessionMemento,
 - mechanizm zapisu i przywracania stanu sesji nauki dla różnych trybów,
 - integrację wzorca z kontrolerami trybów nauki.
- Model domenowy słówek (klasy Word oraz WordSet)
- Ogólna implementacja MVC

-Piotr Borys – klasy ogólne przydatne do działania aplikacji, wzorzec singleton, wzorzec proxy oraz moduł uwierzytelniania użytkownika:

- implementacja klasy Connector, która jest singletonem. Wraz z tym zaimplementowano całą logikę komunikacji z bazą danych,
- implementacja klas ogólnych programu jak ApplicationContext oraz AppRouter,
- implementacja klasy User wraz z AuthenticationController, Autentication oraz LoginPanel z RegisterPanel,
- menu główne, w tym MainMenuController oraz MainMenuPanel,
- implementacja wzorca proxy z klasami WordSetProvider, WordSetLoader oraz WordSetLoaderProxy
- implementacja wzorca state w AppRouter opartego na enumie AppState,
- ogólna implementacja MVC.

PODZIAŁ PRACY W ZESPOLE

- Jan Sztukowski

- implementacja wzorca observer (klasy SessionStatistics, ReviewScheduler),
- zapisywanie i wyświetlanie statystyk po sesji (zaimplementowane w klasach LearningSessionController oraz SessionStatisticsPanel),
- obsługa i wyświetlanie statystyk ogólnych - model (klasa Statistics), view StatisticsPanel) i controller (klasa StatisticsController),
- mechanizm lekcji powtórzeniowych (zaimplementowany w LearningSessionController oraz ReviewScheduler),
- komunikaty o zdarzeniach w trakcie nauki (streak poprawnych odpowiedzi, odpowiednia ilość słów do lekcji powtórzeniowej) zaimplementowane w klasach SessionEventBus i SessionFeedbackBuffer oraz kontrolerach trybów nauki.
- obliczanie i aktualizowanie poziomu językowego użytkownika na podstawie jego statystyk (enum LanguageCEFRLevel oraz metody w User oraz Statistics)
- zestawy słów
- ogólna implementacja MVC